

SECURE-X: A Fresh Approach to Hooking Up Cybersecurity Tools Without Needing a Central Controller That Everyone Trusts

Abstract. We’ve built SECURE-X, a fresh approach to hooking up cybersecurity tools without needing a central controller that everyone trusts. Security systems these days often have tons of monitors – stuff like ransomware detectors, SSH watchers, and web scanners – all feeding info to one main brain. If a hacker gets into that brain, they’re in charge of everything. We’re solving that by using some clever crypto tricks. Our monitors make promises about what they’re seeing but don’t actually show the data. They can even prove their findings match certain security rules without revealing the nitty-gritty details. The main brain just checks these proofs and gives the go-ahead for actions, which the tools then double-check before doing anything. We’ve shown this is rock-solid secure using some fancy math under certain assumptions. We even ran some tests and got response times below 700 milliseconds, which is quick enough to stop ransomware in its tracks. This paper walks you through how we put it all together, proved it’s secure, and checked how well it performs.

Keywords: secure hook-up · fancy math proofs · vector promises · finding threats · crypto stuff

1 What’s the big deal?

1.1 The Problem

We spent a couple of years watching how companies guard their networks, and something seemed off. They all had great security gadgets – stuff to find problems on computers, firewalls, things to spot intruders, and scanners for weak spots. But these tools just didn’t play nice together.

Here’s a classic attack scenario: Someone’s trying to sneak into a server. The SSH thingy sees 50 failed login tries from some weird IP. It jots this down. At the same time, on that same server, the ransomware detector notices a program suddenly making files that look super random – like they’re encrypted. This gets logged too.

The crazy part? These two tools are chilling on the same machine, watching the same attack unfold, but they’re not chatting. The SSH thing doesn’t know about the random files, and the ransomware detector doesn’t know about the login attempts. A real person has to squint at both logs, figure out they’re connected, and then decide to lock down that server. By then? About 500 files are toast.

1.2 SOAR: A Fix That Made Things Worse

The security world tried to deal with this mess with SOAR platforms – basically, ways to automatically hook up security tools and make them react to stuff. The idea was simple: a central brain that gets alerts from all the tools and runs automated actions.

So, an automated action might say: IF (ransomware detector says files are super random) AND (SSH thing says lots of failed logins from the same spot) THEN (firewall locks down that area) AND (scanner checks what’s exposed).

It sounds awesome – until the brain gets hacked. And guess what? That’s been happening. Back in 2021, we saw three cases where hackers went straight for the SOAR platforms. Why? Because that brain sees everything: every IP, every username, every weak spot, every bit of info from every security tool, all in plain sight.

If we were bad guys, we’d go after the brain too. Think about what you could pull off:

1. **Hide Alerts:** The ransomware detector yells “Something’s fishy on this computer!” but you just chuck that message in the trash. Nobody knows anything’s wrong, and the attack keeps rolling.
2. **Fake Alerts:** You make the brain tell the firewall to lock down the CEO’s part of the network. Chaos ensues, and nobody can prove the command wasn’t real.
3. **Steal Info:** Every alert is packed with juicy data. Now you know how the network’s laid out, which computers do what, where the important stuff is, and what security tools are where.
4. **Replay Old Stuff:** You grab real alerts from last month and replay them now. The system freaks out over fake threats, which covers up the real attacks.

We figured things were backwards. We’re trying to beef up security by bringing in a system that actually makes us less secure.

1.3 Our answer: Don’t trust the brain

We asked: Can we make this work even if the brain is totally busted? Not just acting weird, but taken over by a hacker who can mess with anything?

The answer hit us from crypto. What if the monitors never share the real info? Instead, they promise that what they see is true using fancy math – like sealing data in an envelope. Then, they cook up a special proof that says “the stuff in my envelope matches the rules” without ever opening the envelope.

The brain gets these envelopes and proofs. It checks the proofs – yep, this envelope checks out – but doesn’t see what’s inside. If the proof’s legit, the brain signs a command and shoots it over to an enforcer. The enforcer checks the signature before doing anything.

Now, even if attackers own the brain, they can’t: [leftmargin=*]
Sniff what the monitors saw (the envelopes hide the data)
Fake commands (they need real proofs, which need real data that fits the rules)
Hide alerts without getting caught (missing signed commands raise red flags)

1.4 What we did and proved

We made SECURE-X. It has three steps: [leftmargin=*]

Promise: Each monitor grabs its findings (like entropy = 0.96, IP = something, SSH fails = 15) and makes a special math promise. This promise is a short string that ties the monitor to those specific values without showing them.

Prove: Monitors cook up these math proofs showing that their promised values fit the rules. Like, proving “my entropy is super high AND my IP matches the SSH monitor’s IP AND they saw more than 10 failures” without sharing any numbers.

Go: The brain checks the proofs and signs commands. The tools check those signatures and then do their thing.

We showed this is secure using a top-tier method. This means our setup stays safe even when piled on top of other setups doing other things.

We threw together a test version using Rust. The results are good: Basic rules take a blink of an eye, a bit under 70 milliseconds. Complex ones take under 700 milliseconds. Ransomware needs a couple of minutes to scramble files, so we’re golden.

This paper gets into all the bits and pieces. We’ll lay out the math, the security proofs, how the circuits work, and the performance numbers. By the end, you’ll know how to make the whole system work without trusting the brain.

Acknowledgments

We thank our collaborators and the anonymous reviewers for their feedback. This work was supported in part by internal research grants.

References